

Таблиця 5.1 Операції АЛУ

$ALUControl_{1:0}$	Функція
00	Дода
01	Відняти
10	I
11	АБО

5.2.4 ALU

Арифметично-логічний пристрій (ALU) об'єднує різноманітні математичні та логічні операції в одному пристрої. Наприклад, типовий ALU може виконувати операції додавання, віднімання, AND та OR. ALU є серцем більшості комп'ютерних систем.

На [рисунок 5.14](#) показано символ N-бітного ALU з N-бітними входами та виходами. ALU отримує 2-бітний керуючий сигнал $ALUControl$, який визначає, яку функцію виконувати. Сигнали управління зазвичай мають вигляд

показані синім кольором, щоб відрізнити їх від даних. У [таблиці 5.1](#) перелічено типові функції, які може виконувати АЛУ.

На [рисунок 5.15](#) показано реалізацію ALU. ALU містить N-бітний суматор і N 2-вхідних логічних елементів AND і OR. Він також містить інвертори

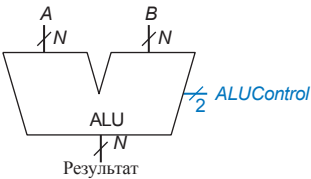


Рисунок 5.14 Символ ALU

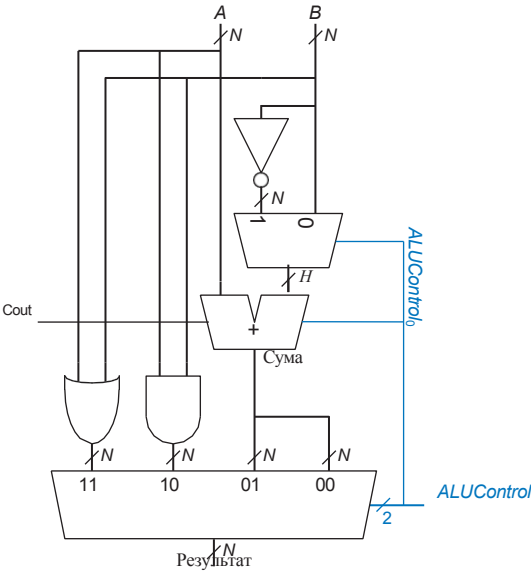


Рисунок 5.15 N-бітний АЛУ

та мультиплексор для інвертування входу B , коли $ALUControl_0$ підтверджено. Мультиплексор 4:1 вибирає бажану функцію на основі $ALUControl$.

Більш конкретно, якщо $ALUControl = 00$, вихідний мультиплексор вибирає $A + B$. Якщо $ALUControl = 01$, ALU обчислює $A - B$. (Згадайте з розділу 5.2.2, що $B + 1 = -B$ в арифметиці двійкового доповнення. Оскільки $ALUControl_0$ дорівнює 1, суматор отримує вхідні дані A і B та підтверджений перенос, що змушує його виконати віднімання: $A + B + 1 = A - B$.) Якщо $ALUControl = 10$, ALU обчислює $A \text{ AND } B$. Якщо $ALUControl = 11$, ALU виконує $A \text{ OR } B$.

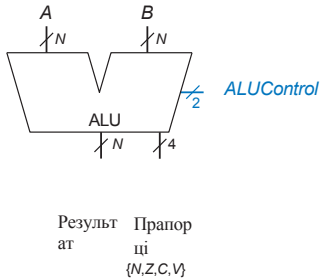


Рисунок 5.16 Символ ALU з прапорцями виходу

Деякі АЛУ генерують додаткові вихідні сигнали, які називаються прапорцями і містять інформацію про вихідні дані АЛУ. На рисунку 5.16 показано символ АЛУ з 4-бітним виходом прапорців. Як показано на схемі цього АЛУ на рисунку 5.17, вихід прапорців складається з прапорців N , Z , C і V , які вказують, відповідно, що вихід АЛУ, *Результат*, є від'ємним або дорівнює нулю, або що суматор створив перенесення або переповнення. Нагадаємо, що найважливішим

Значущий біт числа з доповненням до двох дорівнює 1, якщо воно від'ємне, і 0 в іншому випадку. Таким чином, прапор N (Negative) підключений до найзначущішого непарний біт виходу АЛУ, *Результат*₃₁. Прапор Z (Нуль) встановлюється, коли всі біти *Результату* дорівнюють 0, що виявляється N -бітним логічним елементом NOR на рисунку 5.17. Прапор C (Перенесення) встановлюється, коли суматор виробляє перенесення і АЛУ виконує додавання або віднімання (визначено $ALUControl_1 = 0$).

Виявлення переповнення, як показано на лівій стороні рисунка 5.17, є хитрим ієр. Згадайте з розділу 1.4.6, що переповнення відбувається, коли додавання

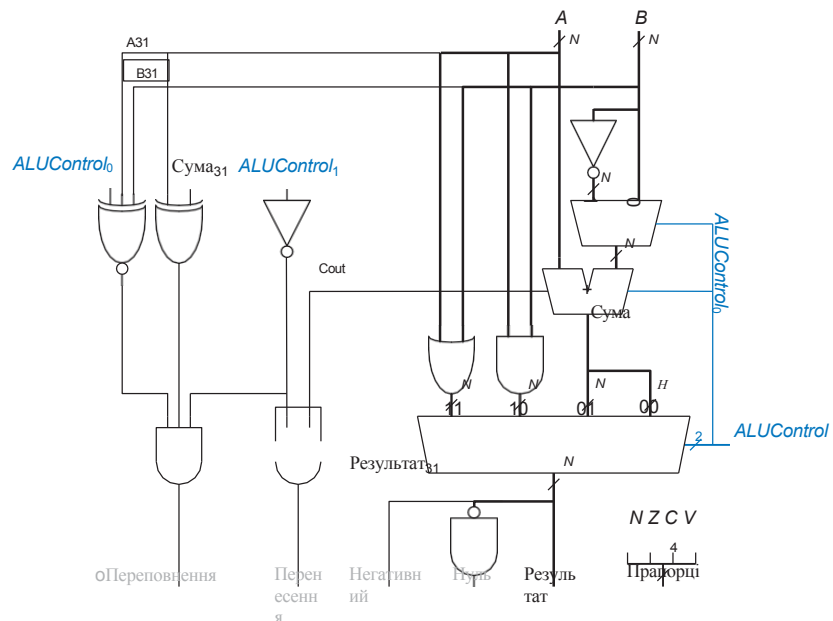


Рисунок 5.17 N-бітний АЛУ з вихідними прапорцями

Таблиця 5.2 Порівняння зі знаком і без знака

Порівняння	Зі знаком	Без знака
$=$	Z	Z
$\neq$	$\overline{Z}$	$\overline{Z}$
$<$	$N \oplus V$	$\overline{C}$
$\leq$	$Z + (N \oplus V)$	$Z + \overline{C}$
$>$	$\overline{Z} \cdot \overline{(N \oplus V)}$	$\overline{Z} \cdot C$
$\geq$	$\overline{(N \oplus V)}$	C

два однакові числа зі знаком дають результат із протилежним знаком. Отже, V (overflow) підтверджується, коли виконуються всі три наступні умови:

- (1) ALU виконує додавання або віднімання ($ALUControl_1 = 0$),
- (2) A і Sum мають протилежні знаки, що виявляється логічним елементом XOR, і
- (3) можливий переповнення. Тобто, як виявляє логічний елемент XNOR, або A і B мають однаковий знак і суматор виконує додавання ($ALUControl_0 = 0$), або A і B мають протилежні знаки і суматор виконує віднімання ($ALUControl_0 = 1$). 3-вхідний логічний елемент AND виявляє, коли всі три умови є істинними, і активує V .

Прапорці ALU також можуть використовуватися для порівнянь, як показано в таблиці 5.2. Щоб порівняти вхідні дані A і B , ALU обчислює $A - B$ і перевіряє прапорці. Якщо Z встановлено, результат дорівнює 0, отже, $A = B$. В іншому випадку A не дорівнює B .

Порівняння величин є складнішим і залежить від того, чи є числа зі знаком або без знака. Наприклад, щоб визначити $A < B$, ми обчислюємо $A - B$ і перевіряємо, чи результат є від'ємним. Якщо числа є без знака, результат буде від'ємним, якщо немає перенесення.¹ Якщо числа мають знак, ми не можемо покладатися на перенесення, оскільки малі від'ємні числа представлені так само, як великі додатні числа без знака. Натомість ми просто обчислюємо $A - B$ і дивимося, чи відповідь є від'ємною, що позначається прапорцем N . Однак, якщо відбувається переповнення, прапорець N буде неправильним. Отже, A менше за B , якщо $N \oplus V$ (іншими словами, якщо відповідь є негативною і немає переповнення або якщо відповідь є позитивною, але відбувся переповнення). Підсумовуючи, визначимо L (сигнал «менше») як істинний, якщо $A < B$. Для чисел без знака $L = C$. Для чисел зі знаком

¹ Ви можете перевірити це, спробувавши кілька чисел. Або ж зверніть увагу, що зміна знака (тобто додавання доповнення до двійки) N -бітних чисел для віднімання дає $-B = B + 1 = 2^N - B$. Тепер $A + (-B) = 2^N + A - B$. Це дасть перенесення (1 у стовпці N), якщо

$A \geq B$ і, отже, перенесення не відбудеться, якщо $A < B$.

числа, $L = N \oplus V$. Решта перевірок є простішими. Менше або дорівнює ($\leq$) — це L АБО Z , оскільки L означає менше, а Z означає дорівнює. Більше або дорівнює ($\geq$) — це протилежність менше: L . Більше ($>$) позначається як більше або дорівнює, але не дорівнює: $L \text{ I } Z$.

Приклад 5.3 ПОРІВНЯННЯ

Розглянемо 4-бітні числа $A = 1111$ і $B = 0010$. Визначте, чи $A < B$, спочатку інтерпретуючи числа як беззнакові (15 і 2), а потім як знакові (−1 і 2).

Розв’язання Обчисліть $A - B = \overline{A} + B + 1 = 1111 + 1101 + 1 = 11101$. Перенесення C дорівнює 1, як показано синім кольором. Прапор N дорівнює 1, як показано курсивом. Прапор V дорівнює 0, оскільки результат має той самий знаковий біт, що й A . Прапор Z дорівнює 0, оскільки відповідь не дорівнює 0000.

Для порівняння без знака $L = \sim C = 0$, оскільки 15 не менше 2. Для порівняння зі знаком $L = N \oplus V = 1$, оскільки −1 менше 2.

Деякі АЛУ також реалізують інструкцію, яка називається «менше ніж» (SLT). Коли $A < B$, *результат* = 1. В іншому випадку *результат* = 0. Це зручно для комп'ютерів, які не мають доступу до прапорців АЛУ, оскільки це, по суті, зберігає інформацію про прапорці в результаті. SLT зазвичай обробляє вхідні дані як підписані. Інший варіант (SLTU) обробляє вхідні дані як непідписані. Існує багато варіацій цього базового АЛУ, які підтримують інші функції, такі як NOT, XOR або XNOR. HDL для N-бітного АЛУ, включаючи версії, що підтримують SLT і прапорці виводу, залишено для прав 5.11–5.14.

Приклад 5.4 РОЗШИРЕННЯ АЛУ ДЛЯ ОБРОБКИ ОПЕРАЦІЇ SLT

Розширити ALU для обробки операції «менше ніж» (SLT).

Рішення Щоб додати ще одну функцію до ALU, ми повинні розширити мультиплексор до п'яти входів. Ми визначаємо, чи A менше за B , виконуючи $A - B$; якщо результат від'ємний, A менше за B . У таблиці 5.3 наведено оновлені сигнали $ALUControl$.

для обробки SLT, а на [рисунку 5.18\(a\)](#) показано розширену схему, де зміни виділено синім (для сигналів керування) та чорним кольорами. Ми використовуємо $ALUControl = 101$ для операції SLT і використовуємо той факт, що встановлення $ALUControl_0 = 1$ змушує суматор виконувати $A - B$. Коли $Sum_{N-1} = 1$, результат $A - B$ є від'ємним, а A менше за B . Отже, ми розширюємо Sum_{N-1} до нуля і подаємо його на вхід мультиплексора 101, щоб завершити операцію SLT. Однак слід зауважити, що ця реалізація не враховує переповнення. Коли відбувається переповнення, Sum матиме неправильний знак. Тому ми виконуємо операцію XOR над бітом знака Sum з V , сигналом переповнення, щоб правильно вказати від'ємне значення Sum , як показано на [рисунку 5.18\(b\)](#).

Таблиця 5.3 Розширена операція ALU для SLT

$ALUControl_{2:0}$	Функція
000	Додавання
001	Відняти
0	AND
011	АБО
10	SLT

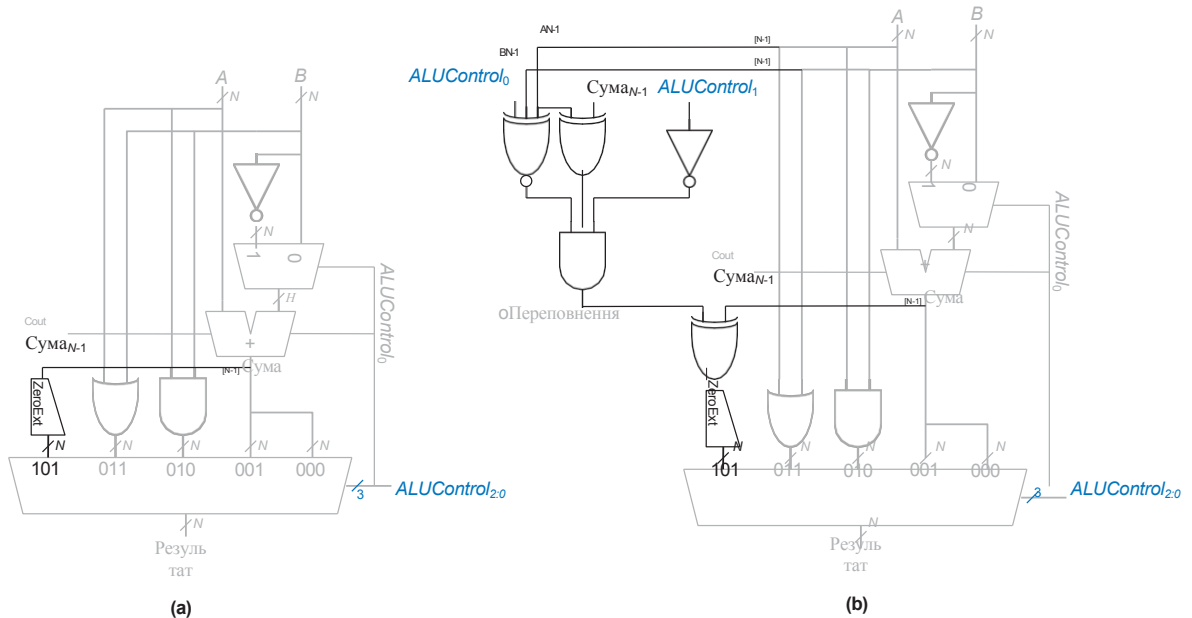


Рисунок 5.18 ALU розширено для підтримки SLT (а) без урахування переповнення, (б) з урахуванням переповнення

5.2.5 Зсувники та ротатори

Зсувні та обертальні пристрої переміщують біти та множать або діляться на степені числа 2. Як випливає з назви, зсувний пристрій зсуває двійкове число вліво або вправо на задану кількість позицій. Існує кілька видів зсувних пристроїв, що часто використовуються:

- **Логічний зсувник** — зсуває число вліво або вправо і заповнює порожні місця нулями.

Приклад: $11001 \gg 2 = 00110$; $11001 \ll 2 = 00100$

- **Арифметичний зсув** — аналогічний логічному зсуву, але при зсуві вправо заповнює найстарші біти копією старого найстаршого біта (msb). Це корисно для множення та ділення чисел зі знаком (див. розділи 5.2.6 та 5.2.7). Арифметичний зсув вліво аналогічний логічному зсуву вліво.

Приклад: $11001 \ggg 2 = 11110$; $11001 \lll 2 = 00100$. Операції $\ll$, $\gg$ та $\ggg$ зазвичай позначають зсув вліво, логічний зсув вправо та арифметичний зсув вправо відповідно.